

TRABALHO DE POO

Sistema de Gerenciamento de Biblioteca

CRUD em Java aplicando os pilares da Programação Orientada a Objetos

INTEGRANTES

Gabriel

Arthur

Maurício

Paulo Henrique

Letícia

UNIVERSIDADE CATÓLICA DE BRASÍLIA

Programação Orientada a Objetos

Prof. Samuel Novais

2026

Tema e Objetivo do Projeto

TEMA

Sistema de Biblioteca

Aplicação Java que gerencia as operações de uma biblioteca: cadastro de autores, livros e usuários, além do controle de empréstimos e devoluções.

Interface: linha de comando (CLI)

Armazenamento: em memória (ArrayList)

Linguagem: Java 25

OBJETIVO

Aplicar os 4 pilares de POO

- Encapsulamento
- Herança
- Polimorfismo
- Abstração

Tudo num sistema funcional com CRUD completo em 4 módulos.

Funcionalidades e Módulos

01 AUTORES

Cadastra autores com nome, nacionalidade e data de nascimento.

Create · Read · Update · Delete

02 LIVROS

Cadastra livros vinculados a um autor (título, ISBN, ano).

Create · Read · Update · Delete

03 USUÁRIOS

Cadastra usuários da biblioteca (nome, endereço, telefone).

Create · Read · Update · Delete

04 EMPRÉSTIMOS

Registra empréstimos e devoluções, atualizando o status do livro.

Create · Read · Update · Devolver · Delete

Diagrama de Classes e Relacionamentos



Setas brancas: HERANÇA (extends) · Losangos: COMPOSIÇÃO (tem-um)

Onde cada conceito aparece no código

01 ENCAPSULAMENTO

Atributos sempre `private` (ou `protected` na classe abstrata) acessados apenas por getters e setters.

ONDE: *Todas as classes do pacote modelo/*

02 HERANÇA

`Autor` e `Usuario` estendem `Pessoa`, reaproveitando atributos `id` e `nome` através de `super(...)`.

ONDE: *Autor.java · Usuario.java*

03 POLIMORFISMO

O método `exibirInfo()` é sobrescrito (`@Override`) em cada subclasse, exibindo formato diferente.

ONDE: *Autor.exibirInfo() · Usuario.exibirInfo()*

04 ABSTRAÇÃO

`Pessoa` é uma classe `abstract` com método abstrato `exibirInfo()` — não pode ser instanciada diretamente.

ONDE: *Pessoa.java*

Classe Pessoa — Abstração e Herança

```
package ucb.poo.biblioteca.modelo;

public abstract class Pessoa {

    protected int id;
    protected String nome;

    public Pessoa(int id, String nome) {
        this.id = id;
        this.nome = nome;
    }

    // método abstrato – cada filha implementa do seu jeito
    public abstract void exibirInfo();

    // getters e setters omitidos
}
```

Java

DESTAQUES

abstract class

Não pode ser instanciada com `new Pessoa()`. Serve como base para `Autor` e `Usuario`.

protected

Atributos visíveis para as subclasses (encapsulamento controlado).

abstract void exibirInfo()

Método sem corpo. Obriga cada subclasse a implementar a sua versão.

Classe Autor — Herança e Polimorfismo

```
public class Autor extends Pessoa {  
  
    private String nacionalidade;  
    private String dataNascimento;  
  
    public Autor(int id, String nome,  
                String nac, String data) {  
        super(id, nome); // chama construtor da Pessoa  
        this.nacionalidade = nac;  
        this.dataNascimento = data;  
    }  
  
    @Override  
    public void exibirInfo() {  
        System.out.println("=== AUTOR ===");  
        System.out.println("Nome: " + this.nome);  
        // ... outras infos  
    }  
}
```

Java

DESTAQUES

extends Pessoa

Autor HERDA id, nome e o método exibirInfo() de Pessoa.

super(id, nome)

OBRIGATÓRIO ser a primeira linha do construtor da subclasse.

@Override

Sobrescreve o método abstrato — POLIMORFISMO. Usuario tem outra implementação.

AutorController — CRUD em ArrayList

```
public class AutorController {  
  
    private ArrayList<Autor> autores;  
    private int proximoId;  
  
    public Autor cadastrar(String nome, ...) {  
        Autor a = new Autor(proximoId, nome, ...);  
        autores.add(a);  
        proximoId++;  
        return a;  
    }  
  
    public Autor buscarPorId(int id) {  
        for (Autor a : autores)  
            if (a.getId() == id) return a;  
        return null;  
    }  
  
    // atualizar(id, ...) e excluir(id) seguem o mesmo padrão  
}
```

Java

PADRÃO REPETIDO

Todos os 4 controllers seguem este mesmo padrão:

```
cadastrar()  
listar()  
buscarPorId()  
atualizar()  
excluir()
```

EmprestimoController tem ainda devolver() — única regra de negócio específica.

Demonstração do fluxo no console

```
Terminal - SistemaBiblioteca

=== SISTEMA DE BIBLIOTECA ===
1 - Autores
2 - Livros
3 - Usuarios
4 - Emprestimos
0 - Sair
Escolha: 4

=== EMPRESTIMOS ===
Escolha: 1
ID do livro: 1
ID do usuario: 1
Data: 30/05/2026
→ Emprestimo cadastrado com ID 1.

=== LIVRO ===
Titulo: Dom Casmurro
Autor: Machado de Assis
Status: Emprestado
```

FLUXO TESTADO

- ✓ Cadastra autor (Machado)
- ✓ Cadastra livro vinculado
- ✓ Cadastra usuário
- ✓ Cria empréstimo
- ✓ Livro vira Emprestado
- ✓ Registra devolução
- ✓ Livro volta a Disponível

Polimorfismo na prática: cada exibirInfo() formata diferente.

CONCLUSÃO

O que o sistema entrega

4

MÓDULOS COM CRUD COMPLETO

Autor, Livro, Usuário e Empréstimo

4

PILARES DE POO APLICADOS

Encapsulamento, Herança, Polimorfismo, Abstração

10

ARQUIVOS JAVA NO PROJETO

5 modelos + 4 controllers + Main

APRENDIZADOS

- Modelar um domínio real (biblioteca) usando classes, herança e abstração.
- Separar responsabilidades em camadas: modelo (dados), controle (CRUD), visão (CLI).
- Implementar regra de negócio (livro fica indisponível ao ser emprestado).

Obrigado!